Documentación técnica — `GestorArit` y `ProcArit`

1) Propósito y resumen

El objetivo es **ejecutar operaciones aritméticas en un proceso externo**.

- El programa principal (`GestorArit`) **lanza** otro programa Java (`ProcArit`) en dos **modos**:
    - `SUM` → suma de dos enteros por línea.
    - `MUL` → multiplicación de dos enteros por línea.
- `GestorArit` **envía** las líneas con datos por la **entrada estándar (stdin)** del proceso, y **lee** los resultados por su **salida estándar (stdout)**.
- `ProcArit` **procesa** cada línea `a b`, imprime el **resultado** y **termina** cuando recibe `FIN`.

---

2) Arquitectura y responsabilidades

Componentes

- **`GestorArit` (proceso padre)**
    - Lanza `ProcArit` con **`ProcessBuilder`**.
    - Envía entradas (`a b` … `FIN`) por **stdin**.
    - Lee salidas (resultados) por **stdout**.
    - Repite el flujo dos veces (modo `SUM` y `MUL`).
- **`ProcArit` (proceso hijo)**

- Valida el **argumento** (`SUM` / `MUL`).
- Lee líneas de **stdin** hasta `FIN`.
- Parseo robusto de dos enteros; si el formato es incorrecto imprime `ERROR`.
- Calcula `a+b` o `a*b` y **escribe** el resultado en **stdout**.

Protocolo de I/O entre procesos

Entrada para `ProcArit` (desde `GestorArit`):

```
 <a> <b>
<a> <b>
...
FIN
```

-
- Salida de `ProcArit`:
  - Una línea por resultado (número) o la palabra `ERROR` si una línea es inválida.

---

3) Detalle por clase

3.1 `GestorArit` (examA.proc.GestorArit)

Puntos clave del código:

- `ProcessBuilder("java","-cp",".","paquete.proc.ProcArit", mode)`
  - Lanza una **nueva JVM** para ejecutar `ProcArit`.
  - `-cp .` indica que las clases están en el **directorio actual** (importante compilar y ejecutar desde la raíz correcta del proyecto).
- `pb.redirectErrorStream(true)`
  - **Fusiona** `stderr` en `stdout` para **simplificar** la lectura (un único `BufferedReader`).

Escritura en **stdin** del hijo:

```
try (BufferedWriter bw = new BufferedWriter(
    new OutputStreamWriter(p.getOutputStream(),
StandardCharsets.UTF_8))) {
    // datos...
    bw.write("FIN");
    bw.flush();
}
```

- Se usa **UTF-8**.
- **Cerrar** el escritor (try-with-resources) señala **fin de entrada** al hijo.

Lectura de **stdout**:

```
try (BufferedReader br = new BufferedReader(
    new InputStreamReader(p.getInputStream(),
StandardCharsets.UTF_8))) {
    while ((line = br.readLine()) != null) out.add(line.trim());
}

     p.waitFor()
```

- Espera a que el proceso termine y obtiene el **exit code**.
- **Nota**: en este diseño `ProcArit` no llama a `System.exit(...)`, así que normalmente devolverá `0` (a menos que se produzca una excepción no capturada).

Responsabilidad del método `run(mode, inputs)`: encapsula el **ciclo completo**: lanzar proceso, escribir dataset, leer resultados, esperar y devolver **todas las líneas de salida**.

3.2 `ProcArit` (examA.proc.ProcArit)

Puntos clave del código:

Validación del **argumento**:

```
if (args.length != 1) { System.out.println("ERROR"); return; }
boolean sumMode = "SUM".equalsIgnoreCase(args[0]);
```

```
boolean mulMode = "MUL".equalsIgnoreCase(args[0]);
if (!sumMode && !mulMode) { System.out.println("ERROR"); return; }
```

- Si el argumento no es válido, imprime `ERROR` y **sale** (sin `System.exit`, por lo que el exit code seguirá siendo 0).

Bucle de **lectura**:

```
while ((line = br.readLine()) != null) {
    line = line.trim();
    if (line.equalsIgnoreCase("FIN")) break;
    if (line.isEmpty()) continue;
    String[] p = line.split("\\s+");
    if (p.length != 2) { System.out.println("ERROR"); continue; }
    ...
}
```

- `FIN` → terminar.
- Línea vacía → ignorar.
- Deben venir **exactamente dos tokens** (dos números).

Cálculo y **salida**:

```
int a = Integer.parseInt(p[0]);
int b = Integer.parseInt(p[1]);
long res = sumMode ? (long)a + b : (long)a * b;
System.out.println(res);
```

- `NumberFormatException` → imprime `ERROR` y **continúa** con la siguiente línea.

---

4) Flujo de ejecución (paso a paso)

`main` en `GestorArit` construye el **dataset**:

```
3 5
10 2
-1 4
```

1. Llama a `run("SUM", datos)`:
    - Lanza `ProcArit SUM`.
    - Escribe todo el dataset + `FIN` por **stdin**.
    - Lee `stdout` completo y lo guarda en una lista.
2. Llama a `run("MUL", datos)` con el **mismo dataset**.
3. Muestra por consola:
    - **Resultados SUM** (cada línea de salida del hijo).
    - **Resultados MUL** (ídem).
    - **ExitCode** de cada ejecución (normalmente 0).

---

5) Contratos de uso y validaciones

- **Entrada esperada (al hijo):** líneas con **dos enteros** separados por espacios.
- **Fin de datos:** la palabra `FIN` en una línea **propia**.
- **Errores de formato:** `ProcArit` **no aborta**, imprime `ERROR` y **sigue** leyendo.
- **Argumento obligatorio:** `SUM` o `MUL` (ignorando mayúsculas). Si no, imprime `ERROR` y termina.

---

6) Ejemplos de ejecución

6.1 Ejecución completa desde `GestorArit`

Salida posible:

```
Resultados SUM:
8
12
3
```

```
Resultados MUL:
15
20
-4
ExitCode SUM: 0
ExitCode MUL: 0
```

6.2 Prueba manual de `ProcArit` (desde consola)

```
> java examA.proc.ProcArit SUM
3 5
10 2
-1 4
FIN
8
12
3
```

(Escribe las líneas y termina con `FIN`)
